

PROJECT JOURNEYS · PRODUCTION ML & AGENTIC AI

Two Platforms, One Discipline: Reliable ML & Multi-Agent AI in Regulated Finance

01 Lending Decision Platform

Google Cloud · Vertex AI · GKE · Argo · Gemini + Claude

02 Fraud Detection Platform

AWS · SageMaker · EKS · EMR/Spark · Airflow · Claude

Staff ML Engineer — interview presentation

What this presentation covers

01

The journey

Discovery → design → development → delivery,
for each platform

02

My contribution

What I personally owned vs. what the team
delivered, mapped to the role

03

Architecture

Full solution diagrams: data, train
agents + RAG

04

Decisions & trade-offs

Why GCP vs AWS tooling, managed vs self-
managed, model & LLM choices

05

Challenges

Regulatory explainability, scale, drift, false
positives — and the fixes

06

Engineering practice

Release management, ML monitor
MLflow / TensorFlow / PyTorch

Same playbook, two clouds, two risk problems

01 LENDING DECISION · Google Cloud

Goal Automate & de-risk credit decisioning; consistent, explainable, compliant lending

Core ML Calibrated gradient-boosted lending-decision model + reason codes

Orchestration Argo Workflows (training DAGs) on GKE

Serving Vertex AI endpoints + agent services on GKE

LLMs Gemini + Claude (task-routed) with RAG over lending policy

Experiments Databricks + MLflow

02 FRAUD DETECTION · AWS

Goal Catch fraud in near-real-time at scale; minimise loss friction

Core ML Hybrid supervised + unsupervised scoring on ext imbalance

Orchestration Airflow (training DAGs); EMR/Spark for feat

Serving SageMaker endpoints + low-latency scoring on EK

LLMs Claude for investigation agents + RAG over case/AM

Experiments Databricks + MLflow

PROJECT 01

Lending Decision Platform

Google Cloud · agentic credit decisioning with explainability and compliance built in



Faster, consistent, defensible credit decisions

Business context

Manual underwriting was slow, inconsistent across officers, and hard to audit. The bank wanted automated decisioning that was fast, consistent, and — critically — explainable and compliant with responsible-lending obligations.

The shape of the solution

A calibrated ML decision model at the core, wrapped in four specialised AI agents that intake the application, run KYC/AML, analyse finances, and audit for security — all grounded in current lending policy via RAG, with a human in the loop for high-value / low-confidence cases.

≤ **minutes**

decision turnaround vs. hours of manual review

Reason codes

on every decision — adverse-action ready & auditable

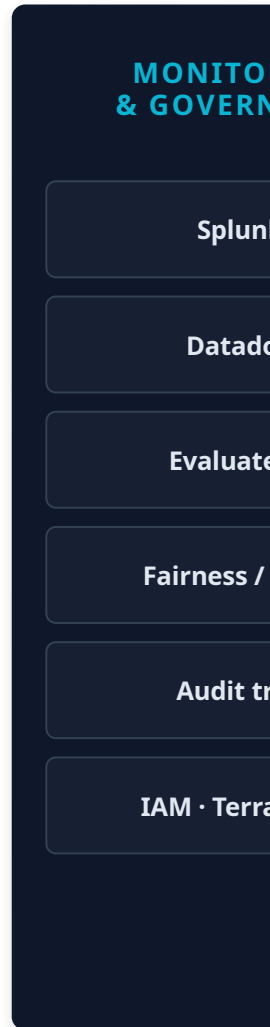
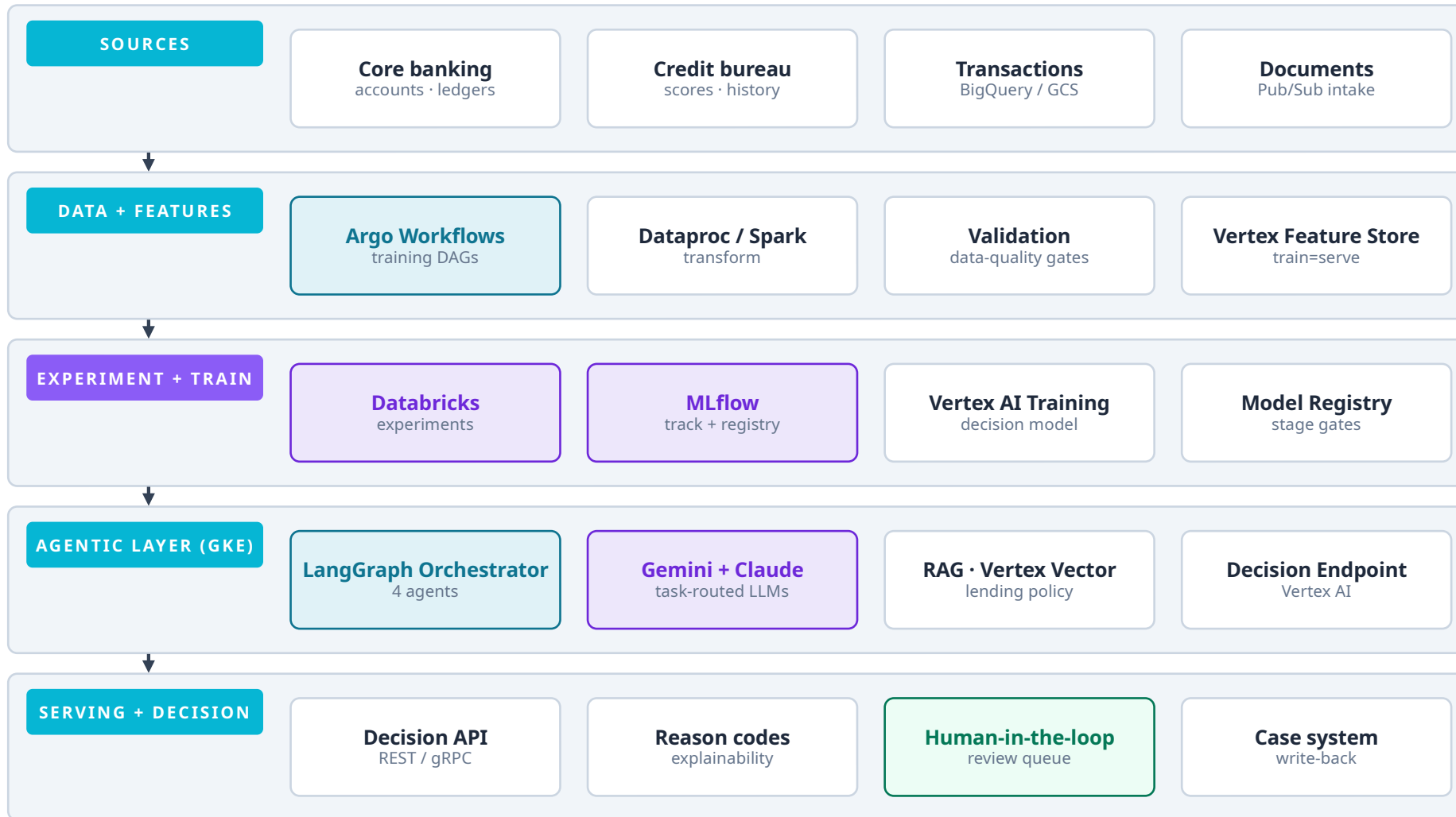
HITL gate

humans review high-value & low-confidence cases

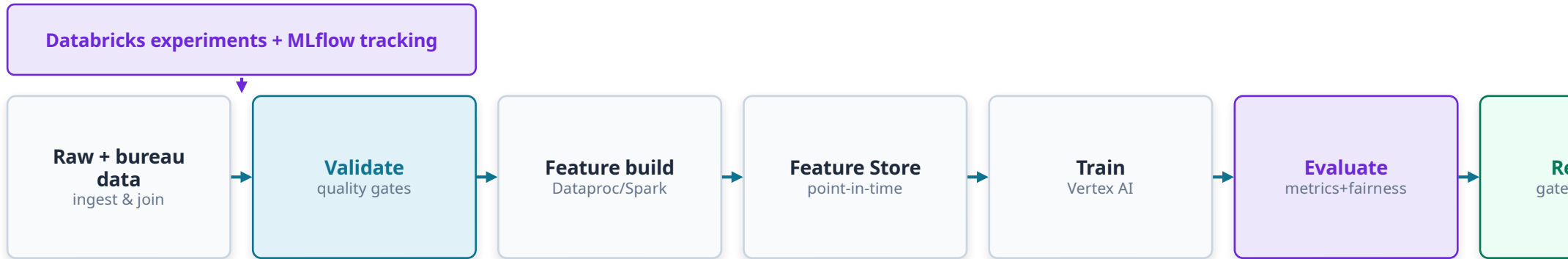
How it unfolded: discovery to delivery



Solution architecture — end to end



How model training works



Orchestrated by Argo Workflows (GKE-native) — every run idempotent, versioned, reproducible

Retraining is triggered, not scheduled by habit. Drift monitors or accumulated label corrections fire a run; the gate blocks promotion unless metric, and calibration checks pass.

Point-in-time features

One definition serves training and the live decision endpoint — no skew, no leakage.

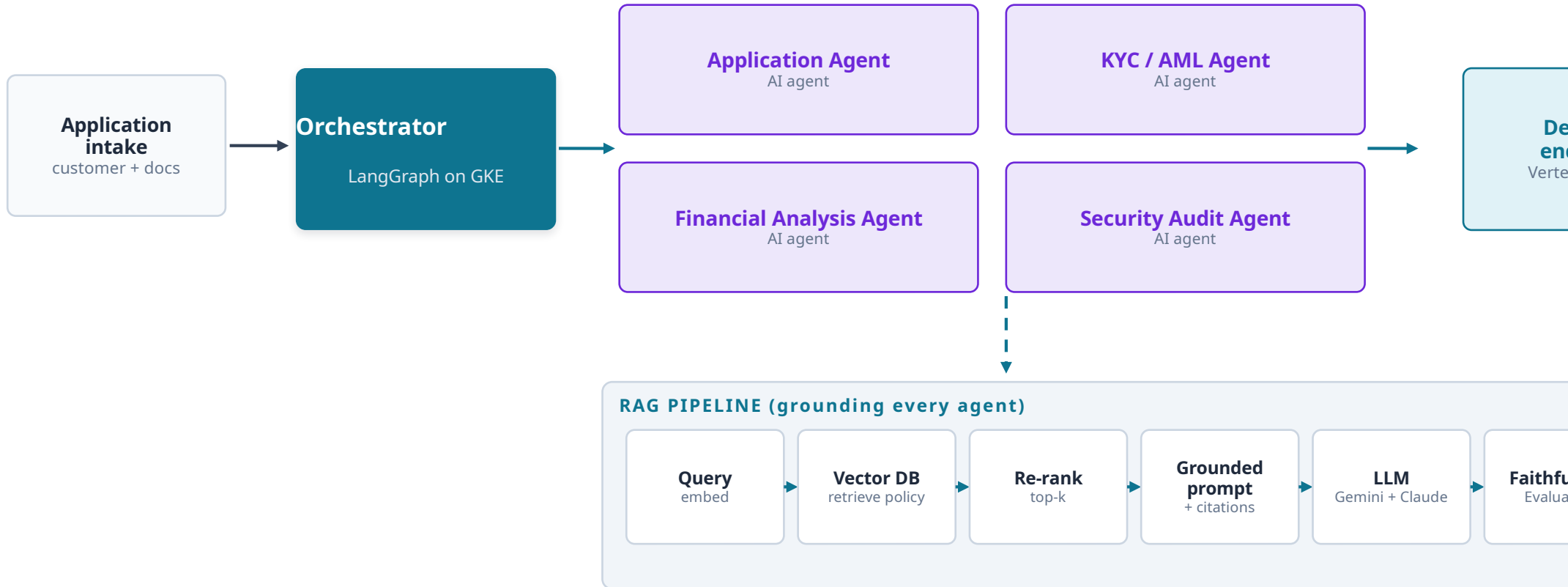
Fairness as a gate

Sliced error + adverse-impact checks run in CI, not as a post-hoc audit.

Calibrated probabilities

Decisions ride on probabilities, so isotonic/PL calibration is mandatory.

AI agents + RAG pipeline



Agents call the ML scoring endpoint + data tools, then return a grounded, cited recommendation → reason codes → human-in-the-loop for the edge case

What I owned vs. what the team delivered

What I personally owned • Staff scope

- Set the technical direction & overall architecture across the platform
- Designed the Argo training pipeline + Vertex Feature Store (train=serve)
- Designed the LangGraph agent orchestration & RAG grounding strategy
- Established the MLOps standards: MLflow registry, CI/CD, release gates
- Owned the model evaluation, fairness & calibration approach
- Mentored 5 engineers; ran the design reviews that set the bar

What the team delivered

- Built source-system ingestion connectors & data contracts
- Implemented individual agent logic against the framework
- Implemented model training code from the pipeline templates
- Ran GKE platform / SRE and on-call operations
- Credit analysts validated decision quality & reason codes
- QA & compliance executed UAT and sign-off

Maps to the role:

System design at scale

Setting standards

Mentoring

MLflow/TF/PyTorch

LLM features

Client & stakeholder intersection



Lending & Credit Risk

Co-owned the decision policy; I translated their rules into model logic, thresholds & reason codes.



Compliance & Legal

Responsible-lending obligations → explainability, fairness testing & audit requirements I designed to.



Model Risk Review

Presented model methodology & v formal sign-off before each promo



Product & Underwriters

Designed the human-in-the-loop review flow with the people who'd actually use it.



Data Governance

Agreed data lineage, access controls & retention for regulated financial data.



Cloud / Google

Architecture reviews on Vertex AI & negotiated managed-service trade

Technology decisions & trade-offs

Decision	What I chose	Trade-off I accepted
Decision model class	Calibrated gradient-boosted trees	Less 'cutting-edge' than deep nets — but explainable, regulator-friendly on tabular data
Serving	Vertex AI managed endpoints	Some GCP lock-in, in exchange for built-in monitoring, safer delivery
Training orchestration	Argo Workflows on GKE	More ops than Vertex Pipelines, bought portability, flexibility
Policy knowledge	RAG over fine-tuning	More moving parts, but fresh, citable policy — essential for compliance defensibility
LLMs	Gemini + Claude, task-routed	Multi-vendor complexity, in exchange for best-fit quality, no single-vendor lock-in

Challenges & how I worked through them

⚠️ Decisions had to be explainable & fair

Interpretable model + SHAP reason codes + adverse-action notices; fairness tested across protected attributes with a full audit trail.

⚠️ LLM hallucination in financial analysis

Grounded every agent via RAG, verified any numeric claim against data, and gated outputs through the Evaluate AI faithfulness harness.

⚠️ Multi-agent latency vs thoroughness

Ran agents in parallel, cached retrievals & embeddings, and held a strict SLA budget with graceful degradation to a simpler path.

⚠️ Disparate, messy source data

Data-quality gates at every boundary + a feature store as the single source of truth, so bad data never reached training or serving.

PROJECT 02

Fraud Detection Platform

AWS · real-time detection at scale on extreme class imbalance, with agentic investigation



Catch more fraud, frustrate fewer customers

Business context

Fraud losses were rising and the existing rules engine was both leaky and noisy — too much fraud got through, while too many genuine customers were wrongly blocked. The mandate: score transactions in near-real-time, at the scale of millions, and cut losses without adding customer friction.

The shape of the solution

A hybrid ML scorer (supervised + unsupervised) on a streaming pipeline, with four investigation agents that triage alerts, run AML checks, analyse finances and audit security — surfacing a grounded case summary to fraud analysts.

~0.1%

fraud base rate — extreme imbalance drove every metric

Near-real-time

sub-second scoring on streaming transactions at scale

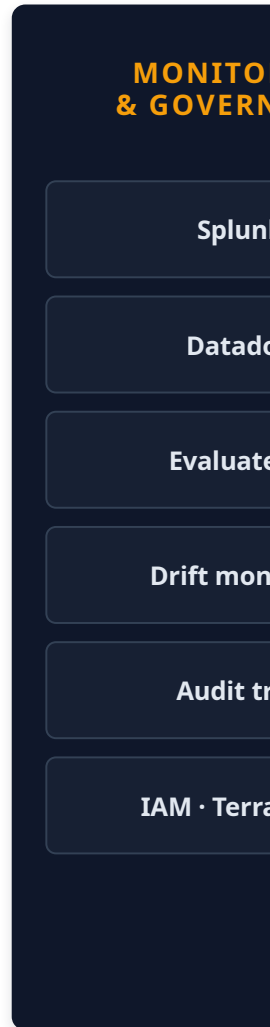
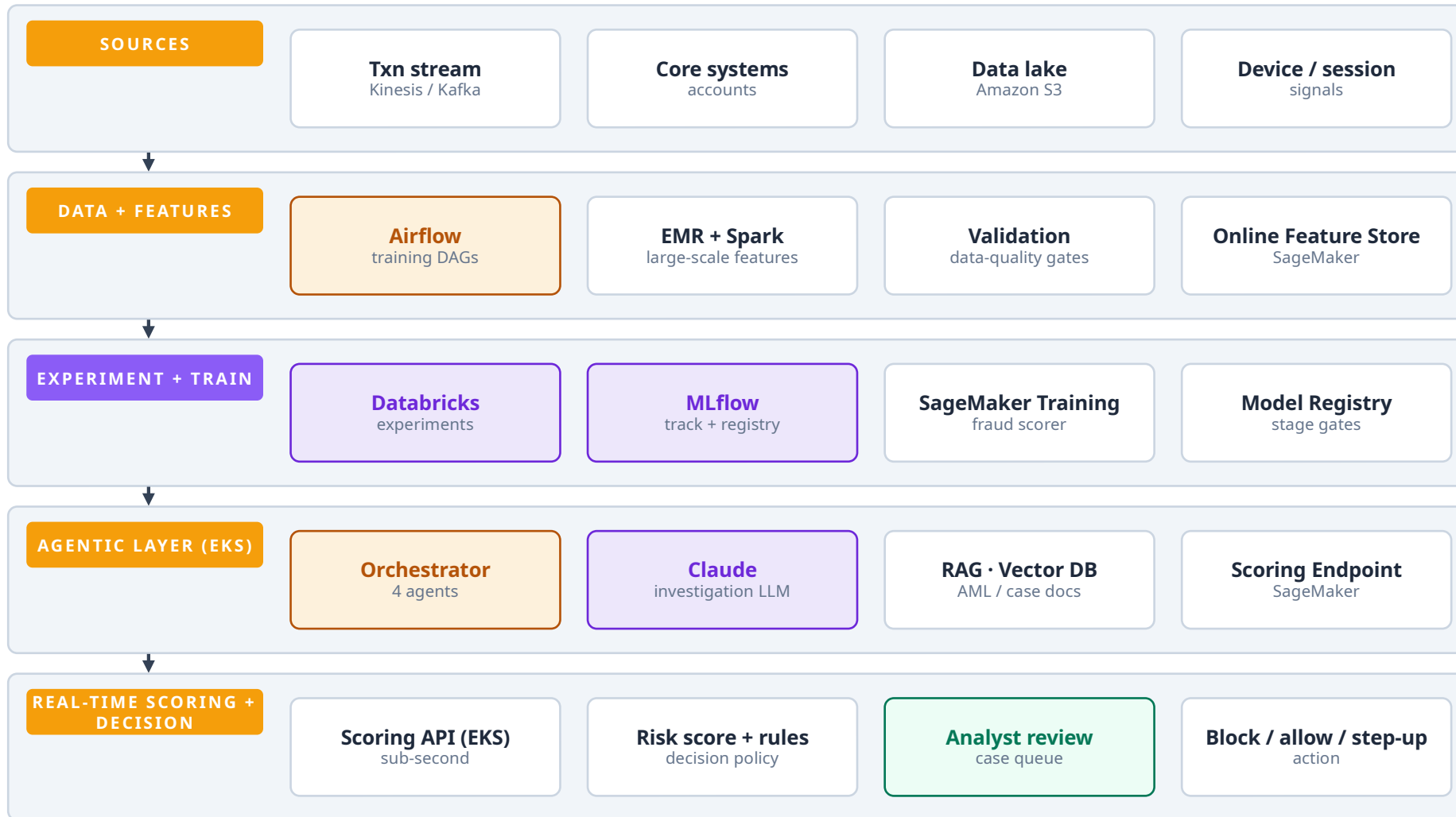
Analyst-in-loop

agents draft the case; humans decide the edge

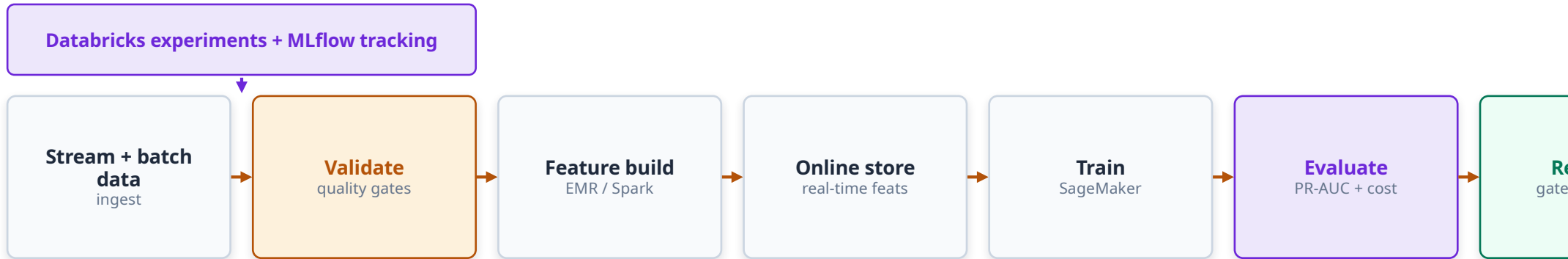
How it unfolded: discovery to delivery



Solution architecture — end to end



How model training works



Orchestrated by Airflow + SageMaker Pipelines — every run idempotent, versioned, reproducible

Retraining is triggered, not scheduled by habit. Drift detectors and the analyst feedback loop trigger retraining; champion–challenger validates a new model in shadow before it ever takes traffic.

Imbalance-aware

PR-AUC & recall-at-fixed-precision, resampling on the train fold only — never raw accuracy.

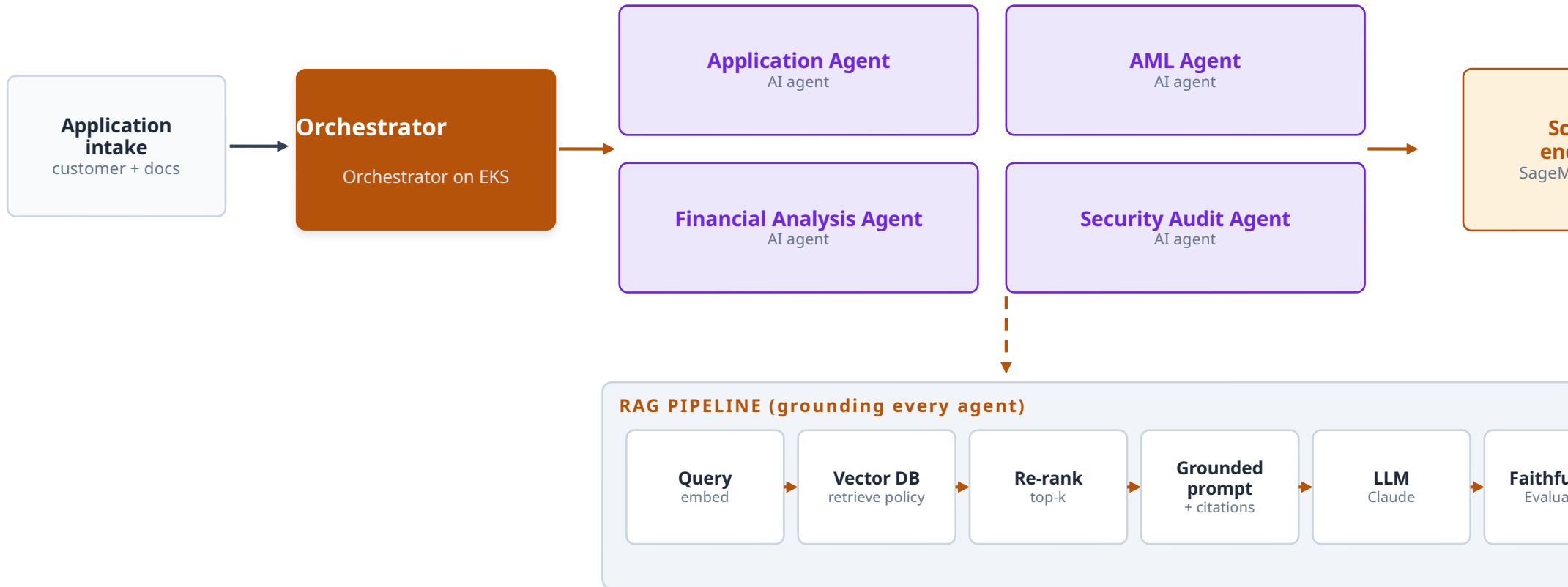
Distributed at scale

EMR/Spark computes billions of behavioural & velocity features; the online store serves them sub-second.

Delayed labels

Fraud is confirmed late, so the pipeline handles delayed & weak labels explicitly.

AI agents + RAG pipeline



Agents call the ML scoring endpoint + data tools, then return a grounded, cited recommendation → reason codes → human-in-the-loop for the edge case

What I owned vs. what the team delivered

What I personally owned · Staff scope

- Owned the end-to-end architecture & the system-design-at-scale decisions
- Designed the EMR/Spark feature pipeline + online feature store
- Set the imbalance-aware evaluation & threshold-tuning methodology
- Designed champion-challenger release & the analyst feedback loop
- Reused & hardened the agent + RAG framework across both platforms
- Mentored engineers; drove the cross-team standards that unified both clouds

What the team delivered

- Built streaming ingestion & Spark feature jobs to spec
- Implemented model training & the agent tools
- Ran EKS platform, autoscaling & on-call
- Fraud analysts labelled cases & tuned review thresholds
- SecOps integrated detection with incident response
- QA executed load & latency testing at scale

Maps to the role:

System design at scale

Spark / distributed

Setting standards

Mentoring

Real-time serving

Client & stakeholder intersection



Fraud Operations

Co-designed the alert queue & case summaries with the analysts who work them daily.



Financial Crime / AML

Aligned detection & agent reasoning with AML typologies and reporting obligations.



Customer Experience

Tuned thresholds against false-positives — protecting genuine customers from fraud.



Risk & Compliance

Presented model validation & monitoring evidence for governance sign-off.



Data Platform / SRE

Agreed streaming SLAs, autoscaling & cost guardrails for real-time scoring.



SecOps

Wired detections into incident response & security audit agent's signals.

Technology decisions & trade-offs

Decision	What I chose	Trade-off I accepted
Model approach	Hybrid supervised + unsupervised	More complexity, but catches both known fraud and day patterns
Feature compute	EMR/Spark over AWS Glue	More cluster ops, bought heavy distributed through instance cost savings
Training & serving	SageMaker managed	Some lock-in, in exchange for managed pipelines, n autoscaled endpoints
Orchestration	Airflow over Step Functions	Self-managed, but richer DAGs & portability across
Decision action	Step-up auth, not just block	More flows to build, but far less customer friction than blocks

Challenges & how I worked through them

⚠️ Extreme class imbalance (~0.1%)

Anchored on PR-AUC & recall-at-fixed-precision tuned to the cost matrix; resampled only inside the training fold; calibrated scores.

⚠️ Sub-second latency at scale

Precomputed heavy features in the online store, computed only live, cached aggressively, autoscaled on EKS with graceful fallback

⚠️ Adversaries & concept drift

Drift monitors + champion-challenger + frequent triggered retraining so the model keeps pace with evolving fraud.

⚠️ False positives hurt real customers

Tuned thresholds to analyst capacity, added step-up auth, and fed decisions back as labels to sharpen precision over time.

PROJECT

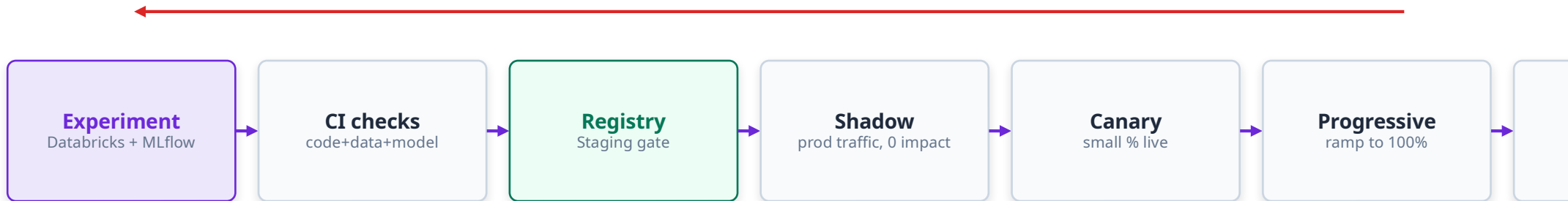
Engineering Practice

The cross-cutting discipline behind both platforms — release, monitoring, and the ML framework strategy



Release management without compromising production

Instant rollback — previous version stays warm



Drift / decay → triggers a new experiment (closed loop)

Every gate is a quality bar

Promotion is blocked unless metric, fairness, calibration & guardrail checks all pass.

Progressive, never big-bang

Shadow → canary → ramp means a bad model is caught on a sliver of traffic, not all of it.

Versioned & traceable

Each prediction maps to a model version, data & feature snapshot — fully auditable & reversible.

Champion-challenge

New models prove themselves against the incumbent before taking real traffic.

Monitoring & measuring ML metrics and accuracy



Operational health

Datadog

Latency p50/p99, throughput, error rate, saturation, cost per 1k predictions



Data & feature drift

PSI / KS monitors

Input distribution shift, prediction-distribution shift, feature freshness & null spikes



Model performance

Sliced metrics

Lending: AUC, calibration, fairness by segment · Fraud: PR-AUC, recall@precision, \$ saved



LLM / agent output

Evaluate AI

Faithfulness, citation, refusal/leak checks on output

Splunk aggregates logs & fires alerts across all four layers. Alert thresholds are tied to business impact, not arbitrary statistics — drift logs & watches, sustained decay triggers retraining, a sharp break rolls back. Ground-truth performance is tracked as labels arrive (immediately for lending, with deliberate lag handling for fraud), so we measure real accuracy, not just proxies.

MLflow, TensorFlow & PyTorch — how they fit together

MLflow — the connective tissue. One tracking + registry layer for both frameworks and both clouds: every run's params, data & versions, metrics and artifacts are logged; the registry governs Staging→Production promotion with the release gates. Framework-agnostic design.

PyTorch · research & iteration

- Fast, flexible experimentation — the default for research & rapid iteration
- Where we explore new architectures and fine-tune transformer / LLM components
- Dynamic graphs make debugging & custom logic straightforward
- Served via TorchServe, or exported to ONNX where a common runtime helps

TensorFlow · production serving

- Mature serving & lifecycle ecosystem (TF Serving, TFX) for production
- Preferred where we need a hardened, high-throughput serving
- Strong fit for managed endpoints on Vertex AI / SageMaker
- Graph optimisation & quantisation for low-latency, low-cost

The strategy in one line: *prototype freely in PyTorch, standardise artifacts & metrics through MLflow, then serve on whichever runtime meets cost & ecosystem needs — gradient-boosted models still win on tabular decisioning, and that's a feature, not a compromise.*

WHAT THESE TWO JOURNEYS SHOW

The through-line

● **Production ML at scale**

Both platforms serve real decisions to real users — feature stores, real-time serving, drift-aware retraining.

● **Principled trade-offs**

Explainable GBMs over flashy deep nets; managed vs self-managed chosen on merit; metrics tied to business cost.

● **Standards across teams**

One agent + RAG pattern, one release & monitoring discipline, reused across two clouds — influence beyond a squad.

● **Bringing people along**

I set the patterns, ran the design reviews, and mentored the engineers who built within them.

Exactly the research-meets-production seam, at scale, for a mission that matters — what I want to do next.